Лабораторная работа.

Микроархитектура GPU.

Оптимизация умножения матриц на модели AMD MI300X в gem5

Содержание

1. Обзор, учебные цели и организационные вопросы . . . 1
1.1 О чём эта работа . . . 1
1.2 Учебные цели и ожидания на защите . . . 1
1.3 О масштабе задачи и о том, чего ожидать . . . 2
2. Почему именно умножение матриц . . . 3
2.1 Операция, на которой держатся вычисления . . . 3
2.2 Почему GEMM может загрузить вычислитель . . . 3
2.3 Как архитектура GPU перестраивается вокруг GEMM . . . 4
2.4 Что из этого есть в нашей модели . . . 5
3. Модель AMDGPU в gem5 и способы запуска программ . . . 6
3.1 Full-system simulation (`full-system`) . . . 6
3.2 rocjitsu + gem5 (`gem5-kmd`) . . . 6
3.3 rocjitsu + simdojo (`simdojo`) . . . 7
3.4 Какой backend выбрать . . . 7
4. Порядок выполнения работы . . . 8
Шаг 1 — Запустите и сохраните `hip-matmul` . . . 8
Шаг 2 — Проследите путь от HIP до инструкций . . . 9
Шаг 3 — Найдите это выполнение в gem5 . . . 9
Шаг 4 — Свяжите статистику с выполнением . . . 10
Шаг 5 — Оцените, насколько быстрым может быть умножение . . . 10
Шаг 6 — Ускоряйте ядро итерациями . . . 11
Шаг 7 — Сдайте ядро, подготовьте отчёт и защиту . . . 12
5. Правила соревнования . . . 13
5.1 Задача и контракт . . . 13
5.2 Метрика и замер . . . 13
5.3 Что можно и чего нельзя . . . 13
5.4 Ориентиры и оценивание . . . 14

1. Обзор, учебные цели и организационные вопросы

1.1 О чём эта работа

Микроархитектуру GPU часто изучают только в теории — по структурным схемам и описаниям системы команд. Эта работа сочетает теорию с практикой: её основой служит исполняемая потактовая модель графического процессора AMD Instinct MI300X, разработанная в AMD Research и входящая в open-source кодовую базу gem5. Открытая модель позволяет познакомиться с реальной ISA и архитектурой GPU заметно глубже: теоретические сведения из учебников и документации проверяются экспериментально, а там, где документация оставляет вопросы, ответ уточняется непосредственно по исходному коду модели — и вы убеждаетесь в правильности собственного понимания.

У работы одна задача для всех — умножение матриц (GEMM, general matrix multiplication). Вы получаете его самую наивную реализацию на HIP C++ и должны сделать её как можно быстрее на модели MI300X. Сначала вы разберёте наивное ядро сверху вниз. Вычислительное ядро (kernel) — это функция, которую хост запускает на GPU сразу в тысячах экземпляров. Вы увидите, во что компилятор транслирует исходный текст на уровне реальной ISA `gfx942`, выполните ядро в gem5 и, обращаясь к исходному коду модели на C++ (`gpu-compute`), проследите, как wavefront выбирается (fetch), планируется, выдаётся на SIMD и приостанавливается (stall) в ожидании памяти. Затем сопоставите эти механизмы с показателями в `stats.txt`, оцените, насколько быстрым ядро может стать в принципе, и начнёте его ускорять.

Ускорение устроено как исследовательский цикл «реализация → измерение → выявление узкого места → изменение кода → повторное измерение» — тот самый, что лежит в основе исследований производительности и совместного проектирования аппаратуры и ПО (hardware/software co-design) в полупроводниковых компаниях. Вы пройдёте его несколько раз и на собственном опыте узнаете, из чего состоит работа исследователя архитектуры. Поскольку задача, машина и метрика у всех одинаковые, результаты сравниваются между собой: работа проходит как соревнование, правила которого даны в разделе 5.

Объект исследования — модель AMD Instinct MI300X (ISA `gfx942`, поколение CDNA3), сконфигурированная с 8 Compute Units и протоколом когерентности Ruby GPU_VIPER.

1.2 Учебные цели и ожидания на защите

Цель работы — сформировать рабочее представление об устройстве и модели программирования GPU, подтвердить его на одном разобранном до конца ядре

ядро HIP → инструкции gfx942 → блоки модели → статистика gem5

и применить это понимание, чтобы найти и снять ограничения производительности умножения матриц.

Готовиться можно как угодно: пользуйтесь любой литературой, любыми инструментами и

ИИ. Оценивается не то, откуда вы взяли знания, а результат: вы отвечаете за корректность отчёта и показываете своё понимание на устной защите.

Включите в отчёт подготовленную вами схему GPU. На защите по этой схеме и материалам отчёта вам нужно:

1. объяснить назначение основных блоков GPU и модель исполнения программы;
2. показать путь наивного ядра, его инструкций и данных;
3. связать выбранные показатели `stats.txt` с тем, что это ядро делает;
4. объяснить, что ограничивает ваше самое быстрое ядро, какие оптимизации вы пробовали, какие из них сработали и почему — включая те, что не сработали.

Знание всей ISA и кодовой базы не требуется. Критерий успеха — способность **самостоятельно объяснить устройство GPU, выполнение своих ядер и сделанные по результатам экспериментов выводы**. Место в рейтинге важно, но засчитывается только результат, который вы можете объяснить.

1.3 О масштабе задачи и о том, чего ожидать

Потактовая симуляция **примерно в миллион раз медленнее реального железа**. Ядро, которое на настоящем MI300X отрабатывает за микросекунды, здесь может считаться от десятков секунд до нескольких минут хост-времени: наивное умножение рейтингового размера занимает около двух минут. Планируйте эксперименты исходя из этого. Корректность проверяйте на функциональном backend'е `simdojo` и на маленьких входах, а в gem5 запускайте версии, про которые уже знаете, что они считают правильно и что вы хотите в них измерить.

Чтобы успешно сделать работу, **не** нужно ни менять, ни пересобирать симулятор: достаточно читать исходный код модели. Более того, для соревнования конфигурация модели фиксирована — всё ускорение должно прийти из вашего кода.

2. Почему именно умножение матриц

2.1 Операция, на которой держатся вычисления

Произведение матриц $C = A \cdot B$, в котором каждый элемент результата — скалярное произведение строки A на столбец B, выглядит учебным примером, но на нём держится значительная часть современных вычислений.

В нейронных сетях к нему сводятся практически все дорогие операции. Полносвязный слой умножает матрицу активаций на матрицу весов. В трансформере это проекции запросов, ключей и значений, выходная проекция и двухслойный MLP каждого блока, а само внимание состоит из двух батчевых умножений: $QK^\top$ и произведения весов внимания на V. Свёртки обычно выполняются как неявное умножение матриц (implicit GEMM). Обратное распространение ошибки добавляет к каждому прямому умножению ещё два — для градиентов по входу и по весам. В итоге при обучении и инференсе больших моделей на GEMM приходится подавляющая часть операций с плавающей точкой, и их стоимость почти целиком определяется тем, насколько быстро аппаратура умножает матрицы.

Так же устроены научные вычисления. Библиотеки плотной линейной алгебры — LAPACK и его GPU-аналоги — выполняют LU-, QR- и разложение Холецкого блочными алгоритмами именно для того, чтобы основная работа пришлась на умножение блоков. Тест HPL, по которому составляется рейтинг суперкомпьютеров TOP500, решает плотную систему линейных уравнений и почти всё время проводит в умножении матриц двойной точности. К GEMM сводятся расчёты электронной структуры в квантовой химии, многие задачи обработки сигналов и изображений, рекомендательные системы. Поэтому ускорение умножения матриц даже на несколько процентов ощутимо сразу во многих областях.

2.2 Почему GEMM может загрузить вычислитель

Произведение двух матриц $n \times n$ требует $2n^3$ операций, а читает и пишет всего $3n^2$ чисел: каждый элемент A и B участвует в n умножениях. Отношение числа операций к объёму данных — **арифметическая интенсивность** — растёт с размером задачи. Большинство других алгоритмов, от сложения векторов до гистограмм, выполняют на каждый загруженный байт фиксированное небольшое число операций и упираются в пропускную способность памяти, как бы быстро ни считали ALU. GEMM — редкое исключение: если данные переиспользовать, его можно сделать ограниченным вычислениями, а не памятью.

Это «если» и составляет суть оптимизации. Наивная реализация, с которой вы начнёте, переиспользования не извлекает: на каждое умножение с накоплением она запрашивает из памяти два числа. Быстрые реализации разбивают задачу на тайлы и поднимают их по иерархии памяти — из HBM в кэши, из кэшей в локальную память workgroup'а (LDS), из LDS в регистры — так, чтобы каждый загруженный элемент использовался много раз на ближайшем к ALU уровне. Одновременно загрузку следующего тайла совмещают с вычислениями над текущим, чтобы вычислитель не простаивал в ожидании данных. Все эти приёмы вам предстоит попробовать и измерить.

2.3 Как архитектура GPU перестраивается вокруг GEMM

Вычислительная мощность кристаллов растёт быстрее пропускной способности памяти, и GEMM — одна из немногих массовых задач, способных эту мощность использовать. Поэтому последние поколения ускорителей наращивают производительность прежде всего за счёт специализированных матричных блоков: у NVIDIA это тензорные ядра (Tensor Cores), у AMD — матричные ядра (Matrix Cores), исполняющие инструкции MFMA (matrix fused multiply-add). Одна такая инструкция перемножает небольшие блоки матриц силами всего wavefront'а: сотни и тысячи умножений с накоплением за одну выдачу вместо одного на lane. Это экономит не только такты, но и энергию на выборку инструкций и чтение регистрового файла, которая у обычного векторного FMA сравнима со стоимостью самой арифметики. Специализированные ускорители вроде Google TPU идут ещё дальше и строятся почти целиком вокруг систолического массива для умножения матриц.

Второе направление — точность. Площадь и энергия умножителя растут примерно квадратично с шириной мантиссы, а объём передаваемых данных — линейно с шириной числа. Нейросети терпимы к низкой точности, поэтому им выгодны узкие форматы: сначала FP16 и BF16, затем FP8, теперь FP6 и FP4. Чтобы при столь малом диапазоне значения не терялись, новые форматы используют блочное масштабирование (microscaling, спецификация OCP MX): небольшой блок чисел хранит общий масштабный множитель. Накопление при этом обычно остаётся в FP32.

Третье направление — доставка данных. Матричный блок настолько быстр, что главной трудностью становится вовремя подать ему операнды, и аппаратура постепенно забирает эту работу у программы:

- **NVIDIA Hopper** (H100, 2022): тензорные ядра четвёртого поколения с форматом FP8 и Transformer Engine, управляющим его масштабированием; асинхронные матричные инструкции `wgmma`, которые исполняет группа из четырёх warp'ов и которые берут операнды прямо из разделяемой памяти; Tensor Memory Accelerator (TMA) — блок, копирующий многомерные тайлы из глобальной памяти в разделяемую без участия потоков; кластеры блоков потоков с распределённой разделяемой памятью.
- **NVIDIA Blackwell** (B200, 2024): тензорные ядра пятого поколения с форматами FP6 и FP4, аппаратной поддержкой microscaling и собственным форматом NVFP4; отдельная тензорная память (Tensor Memory) для аккумуляторов рядом с тензорными ядрами, освобождающая регистровый файл; матричные операции, которые запускает один поток и которые могут охватывать пару SM.
- **NVIDIA Rubin** (2026) продолжает ту же линию: память HBM4 и дальнейший рост производительности тензорных ядер в низкоточных форматах, прежде всего NVFP4.
- **AMD CDNA**: инструкции MFMA появились в первом поколении CDNA (MI100, 2020); CDNA2 (MI200) добавила матричные операции FP64 для HPC; CDNA3 (MI300X, 2023) — FP8 и структурированную разреженность 2:4. CDNA4 (MI350/MI355X, 2025) добавила FP6 и FP4 с MX-масштабированием, удвоила производительность матричных ядер в низкой точности и увеличила LDS до 160 КиБ, чтобы на кристалле помещались тайлы большего размера.

Общая картина такова: всё большая часть кристалла отводится матричным блокам, всё больше логики — доставке к ним данных, а задача программиста, пишущего GEMM, — сделать так, чтобы данные приходили вовремя и использовались повторно.

2.4 Что из этого есть в нашей модели

MI300X относится к поколению CDNA3, и модель gem5 исполняет его инструкции MFMA — с операндами FP32, FP16, BF16, FP8, INT8 и FP64. Матричные ядра вам доступны через встроенные функции компилятора `_builtin_amdgcn_mfma*`. Аналогов TMA и тензорной памяти в CDNA3 нет: данные доставляет сама программа — глобальными загрузками, через LDS и регистры. Именно эту доставку вам и предстоит строить.

Учебная конфигурация отличается от настоящего MI300X не только числом CU: вместе с ними уменьшена и подсистема памяти. Поэтому баланс между вычислениями и памятью здесь другой, чем на реальном ускорителе, и какой из пределов ограничивает ваше ядро, нужно не предполагать, а оценить (шаг 5) и измерить. В соревновании умножение выполняется в FP32; почему и как быть с низкой точностью, объясняется в разделе 5.

3. Модель AMDGPU в gem5 и способы запуска программ

В этой работе HIP-программа выполняется не на физическом GPU, а на его потактовой модели в gem5. Целевая конфигурация использует ISA `gfx942`, 8 Compute Units и подсистему памяти Ruby GPU_VIPER. Уменьшенное число CU сокращает время симуляции и сохраняет возможность исследовать инструкции, регистры, LDS и кэши. Объёмы регистров, число wavefront'ов и локальные кэши CU соответствуют документации MI300X, а общий L2 уменьшен до 1 МиБ. Тактовая частота GPU фиксирована на 2,1 ГГц; это выбранная рабочая точка, а не модель динамического boost. Источники параметров и оставшиеся приближения задержек собраны в аудите конфигурации MI300X.

Получив пакет диспетчеризации ядра, модель создаёт workgroup'ы и wavefront'ы и распределяет их по Compute Units. Затем она моделирует выдачу и выполнение инструкций, работу регистров и LDS, обращения к TLB, кэшам и памяти. Параметры созданной машины сохраняются в `config.ini`, а счётчики её работы — в `stats.txt`. Именно по этим данным вы будете объяснять поведение ядра.

Прежде чем ядро попадёт на GPU, программная среда должна выделить память, создать очередь и передать пакет AQL командному процессору. Эту часть можно устроить по-разному, поэтому скрипты лабораторной работы предлагают два backend'а gem5 и один вспомогательный функциональный backend.

3.1 Full-system simulation (`full-system`)

gem5 загружает Linux, и программа использует обычный стек ROCm и гостевой драйвер `amdgpu`/KFD:

```
HIP-приложение → ROCm → amdgpu/KFD → модель AMDGPU
```

Это официальный путь, поддерживаемый AMD и проверенный на реальных нагрузках: модель GPU здесь работает ровно в том окружении, для которого её разрабатывали. Требует x86-64 и KVM, а также места на диске под образ гостевой системы.

3.2 rocjitsu + gem5 (`gem5-kmd`)

Гостевая Linux-система не запускается. rocjitsu передаёт операции KFD/DRM серверу HSAKMT, встроенному в gem5, а тот настраивает модель AMDGPU и передаёт ей пакеты AQL:

```
HIP-приложение → ROCr → rocjitsu → HSAKMT service → модель AMDGPU
```

Эту интеграцию сгенерировал ИИ-агент специально для лабораторной работы: без гостевой системы отпадает требование KVM и x86-64, поэтому работу можно делать даже на ARM-ноутбуке и с заметно меньшим объёмом диска. Но этот путь совсем не обкатан: вы будете

его первыми пользователями, так что баги вполне вероятны.

3.3 rocjitsu + simdojo (`simdojo`)

Приложение использует ROCr и interposer rocjitsu, а инструкции GPU исполняет функциональный симулятор simdojo в процессе приложения:

```
HIP-приложение → ROCr → rocjitsu → simdojo
```

gem5 здесь не участвует, поэтому `stats.txt` не появляется и измерять на этом backend'е нечего. Зато он на порядки быстрее и исполняет ту же ISA `gfx942` — используйте его, пока отлаживаете корректность ядра.

3.4 Какой backend выбрать

Оба backend'а gem5 используют одну модель конвейера GPU и отличаются настройкой платформы и программным путём до неё, поэтому сравнивать версии ядра можно только внутри одного backend'а. Рейтинг соревнования считается на `gem5-kmd`: он работает на любой машине, и на нём же удобнее всего вести все измерения. `full-system` пригоден для разбора наивного ядра и для проверки выводов в «настоящем» окружении, но его цифры с рейтинговыми не смешивайте. `simdojo` остаётся вспомогательным: на нём быстро проверяют, что ядро вообще считает правильно. Единый порядок сборки, запуска и выбора backend'а приведён в руководстве Running the GPU lab.

4. Порядок выполнения работы

Работа идёт от полностью разобранного наивного ядра к собственной быстрой реализации:

Шаги	Что вы делаете
1–4	разбираете наивный `hip-matmul` от исходника на HIP до статистики gem5
5	оцениваете, насколько быстрым умножение может быть на этой машине
6	итерациями ускоряете ядро, проверяя каждую гипотезу измерением
7	сдаёте ядро на рейтинговый замер, готовите отчёт и защиту

Разбор наивного ядра — не формальность перед «настоящей» работой. Именно в нём вы научитесь читать ISA, трассы и статистику, без которых оптимизация превращается в перебор наугад.

Шаг 1 — Запустите и сохраните `hip-matmul`

Соберите пример под `gfx942` и добейтесь `PASS` на выбранном backend'е gem5. Исходник примера перемножает матрицы `A` размера `M × K` и `B` размера `K × N`, получая `C` размера `M × N`. По умолчанию `M = 256`, `N = 256`, `K = 128` — это и есть рейтинговый размер. Алгоритм самый простой: один work-item считает один элемент результата и читает для этого целую строку `A` и целый столбец `B` прямо из глобальной памяти.

Эта реализация **намеренно неэффективна**. Каждый элемент `A` запрашивается `N` раз, а каждый элемент `B` — `M` раз: на одно умножение с накоплением приходятся две глобальные загрузки. Кэши могут обслуживать повторные обращения, поэтому число загрузок не равно числу чтений из HBM. Пример позволяет увидеть в статистике работу конвейера памяти, ожидания `s_waitcnt` и роль кэшей, а затем исследовать, как явное переиспользование данных меняет эти показатели.

Найдите в исходнике ядро `matmul`, заполнение матриц, геометрию запуска и сверку результата с хостовым эталоном. Второе ядро файла, `matmul_transposed_b`, понадобится на шаге 4. Затем посмотрите в Makefile примеров, как `hipcc` собирает программу под `gfx942` и зачем используется `--save-temps`.

Программа печатает контрольную сумму. Для выбранного заполнения значениями `0…7` и размеров, кратных восьми, она равна `49·M·N·K/4`: при каждом значении индекса редукции средние значения по строкам `A` и столбцам `B` равны `3.5`. Для исходных размеров контрольная сумма — `102 760 448`. Пропущенное или неверно посчитанное ядро видно по выводу сразу.

После сборки рабочие файлы находятся под `gpu-lab/workspace/work/`:

- `apps/<backend>/hip-matmul` — исполняемая HIP-программа;
- `apps/<backend>/save-temps/hip-matmul/` — промежуточные файлы компилятора;
- `runs/hip-matmul-<backend>-*/` — каталоги выполненных запусков.

Оба backend'а gem5 сохраняют `stats.txt`, `config.ini`, `config.json` и `gem5.log`; у `simdojo` этих файлов нет. При full-system-запуске строку `PASS` обычно ищите в `system.pc.com_1.device`, а в двух rocjitsu-вариантах — в `app.stdout`. Не переходите дальше, пока не сможете объяснить назначение файлов выбранного backend'а и пока повторный запуск не даст `PASS`. Сохраните этот прогон: он станет точкой отсчёта для всех ваших оптимизаций.

Шаг 2 — Проследите путь от HIP до инструкций

В исходном запуске сетка состоит из 16×16 = 256 workgroup'ов по 16×16 work-item'ов; на `gfx942` это четыре wavefront'а на workgroup и 1024 wavefront'а на всю сетку. Проверьте это сначала по `main.cpp`, а затем откройте сгенерированный компилятором файл `gpu-lab/workspace/work/apps/<backend>/save-temps/hip-matmul/main-hip-amdgcn-amd-amdhsa-gfx942.s`. В нём найдите ассемблерный код ядра — фрагмент между меткой `_Z6matmulPKfS0_Pfjjj` и `s_endpgm`. Разберите все инструкции этого фрагмента.

Сначала разделите фрагмент на три части: пролог (загрузка аргументов ядра, вычисление `row` и `col`, проверка выхода за границу), цикл по `k` и эпилог (запись `c[row * n + col]`). Обратите внимание, что обе проверки границы компилятор свёл к одному сравнению вида `max(row, col) < n`, а его результат положил в маску `EXEC`.

Главное здесь — тело цикла, чуть больше десятка инструкций: две `global_load_dword`, пересчёт адресов и счётчика, `s_waitcnt vmcnt(0)`, `v_fmac_f32` и переход назад. Ровно эта последовательность и делает ядро неэффективным: на одно умножение с накоплением приходятся две загрузки из глобальной памяти, а `s_waitcnt` перед ним останавливает wavefront до их завершения. Посчитайте по листингу, сколько раз тело цикла выполняется в одном wavefront'е и сколько загрузок это даёт на всю сетку.

Затем по справочнику CDNA разберите каждую инструкцию: какие значения находятся в SGPR и VGPR, как маска `EXEC` реализует условие и что именно считает `s_waitcnt`. После кода просмотрите дескриптор `.amdhsa_kernel`, блок `Kernel info` и `.amdgpu_metadata`: в них указаны используемые регистры, LDS и размещение аргументов ядра. Директивы секций и выравнивания не относятся к алгоритму. Цель — научиться восстанавливать по ISA исходное вычисление и зависимости данных, а не выучить систему команд целиком. Этот навык понадобится на каждой итерации шага 6: только по ISA видно, что компилятор на самом деле сделал с вашей оптимизацией.

Шаг 3 — Найдите это выполнение в gem5

По имени ядра найдите его dispatch и размеры grid/workgroup. Затем проследите тот же путь в модели: от диспетчеризации workgroup до fetch, проверки готовности wavefront, планирования, исполнительного блока и памяти.

Трасса `GPUExec` печатает каждую выполненную инструкцию каждого wavefront'а, поэтому на полном запуске она неподъёмна. Снимайте её на уменьшенном входе: `--size 16` даёт одну workgroup, четыре wavefront'а и шестнадцать итераций цикла — этого хватает, чтобы сопоставить фактическую последовательность инструкций с ассемблерным листингом и увидеть, как повторяется тело цикла. Как передать приложению аргументы и включить трассу, показано в Running the GPU lab.

Начните схему GPU и нанесите на неё этот путь, SGPR/VGPR, маску `EXEC`, основные конвейеры CU и уровни памяти. Схема должна расти вместе с вашим пониманием, а не рисоваться по памяти перед сдачей.

Шаг 4 — Свяжите статистику с выполнением

Выберите несколько показателей, которые можно объяснить через код и трассу: инструкции и активные lane, работу ALU и памяти, ожидания, LDS и кэши. Для каждого вывода сформулируйте причинную цепочку «HIP → ISA → блок модели → показатель». Вся статистика уровня CU идёт под `system.cpu1.CUs<N>.` в `stats.txt` и под `system.Shader.CUs<NN>.` в `config.ini` — ищите по обоим префиксам.

Хорошая отправная точка — доля тактов, в которые CU не выдал ни одной инструкции (`ExecStage.numCyclesWithNoIssue` против `totalCycles`). Она велика, но прежде чем списывать её на `s_waitcnt` из шага 2, проверьте, чем CU был занят. На 8 CU приходится в среднем по 32 workgroup'а, однако их начало и завершение не обязаны совпадать: отделяйте ожидания активных wavefront'ов от отсутствия работы на CU. Затем сравните, сколько тактов ушло на `VectorALU0`, `VectorMemPipe` и `SharedMemPipe`, и сойдётся ли соотношение с «две загрузки на одно умножение».

Дальше поставьте контрольный опыт. В примере есть второе ядро, `matmul_transposed_b`: оно считает то же произведение, но `B` хранится по столбцам, поэтому соседние lane одного wavefront'а читают адреса не подряд, а с шагом `K` элементов. Запустите пример с `--transpose-b` и сравните два прогона. Вычисление, объём данных и геометрия запуска совпадают, так что вся разница относится к паттерну доступа: смотрите на число обращений к L1 (`tcp`), на hit rate L1 и L2 (`tcc`) и на такты в конвейере памяти. Заранее очевидного ответа здесь нет — «неудобный» паттерн меняет сразу и объём обращений, и локальность, и объяснить нужно обе стороны. Этот опыт — первая подсказка к соревнованию: одно и то же вычисление может стоить очень по-разному в зависимости от того, как данные идут через память.

Сравнивать прогоны нужно по суммам показателей всех 8 CU: раскладка workgroup'ов по CU от запуска к запуску меняется, поэтому отдельно взятый `CUs0` в двух прогонах несопоставим, даже если это два прогона одной и той же команды.

Помните, что в `full-system` вызовы `hipMemcpy` могут запускать служебные blit-ядра, поэтому не приписывайте весь `stats.txt` одной функции. Укажите и границы интерпретации: восемь CU — учебная конфигурация, а не полный XCD или MI300X. В `gem5-kmd` топология ROCr соответствует числу моделируемых CU; в текущем full-system guest HIP сообщает 320 CU, хотя модель исполняет работу на восьми. Для исследования масштабирования проверяйте `config.ini`, а не только свойства устройства из HIP.

Шаг 5 — Оцените, насколько быстрым может быть умножение

Прежде чем оптимизировать, выясните, куда вообще можно дойти. Ядро не может быть быстрее двух пределов: времени, за которое машина выполнит все его операции, и времени, за которое она передаст минимально необходимые данные.

1. Посчитайте работу: `2·M·N·K` операций с плавающей точкой.
2. Оцените вычислительный предел. По `config.ini` определите число CU, SIMD и частоту, по справочнику ISA и исходному коду модели — сколько FP32-умножений с накоплением за такт выполняет векторный ALU и сколько — матричное ядро на одной инструкции MFMA с операндами FP32. Таблица тактов, которые модель отводит на каждую инструкцию MFMA, находится в `src/gpu-compute/compute_unit.cc`.
3. Оцените предел по памяти. Минимальный трафик — прочитать `A` и `B` по разу и один раз записать `C`. Пиковую пропускную способность памяти возьмите из `config.ini`, а затем сравните её с тем, что показывает статистика памяти наивного прогона: пик не обязательно достижим.
4. Нанесите оба предела и наивное ядро на диаграмму roofline — производительность против арифметической интенсивности — и определите, какой предел здесь главный и во сколько раз наивное ядро от него отстаёт.

Результат этого шага — ориентир для всей оптимизации: он подсказывает, какие приёмы могут окупиться, а какие заранее упрутся в другой предел. Не удивляйтесь, если выводы разойдутся с тем, что вы знаете о настоящем MI300X: причина в учебной конфигурации (раздел 2.4), и объяснить это расхождение — часть задания.

Шаг 6 — Ускоряйте ядро итерациями

Возьмите за основу `main.cpp` из `hip-matmul` и меняйте ядро и геометрию его запуска, соблюдая контракт из раздела 5. Каждая итерация проходит один и тот же цикл:

1. запишите гипотезу до изменения кода: какой механизм вы меняете, какой показатель должен сдвинуться и почему;
2. сделайте одно изменение и добейтесь `PASS` на `simdojo`, сначала на маленьком входе;
3. проверьте по ISA, что компилятор сделал то, что вы задумали;
4. выполните версию в gem5 и сравните её с предыдущей по рейтинговой метрике и по нескольким контрольным показателям;
5. запишите результат в журнал экспериментов, даже если гипотеза не подтвердилась.

Журнал — таблица версий: что изменено, какая была гипотеза, GPU-такты, ключевые показатели и вывод. Неудачные попытки в нём так же ценны, как удачные: объяснённое «не сработало, потому что…» — полноценный результат и хороший материал для защиты.

Направлений для оптимизации много, и перечисленные ниже — карта, а не рецепт. Порядок, сочетание и конкретные параметры определяйте по собственным измерениям.

Переиспользование в регистрах. Пусть work-item считает не один, а несколько выходных элементов: один загруженный операнд тогда участвует в нескольких накоплениях. Широкие загрузки (`global_load_dwordx4`) сокращают число инструкций памяти. Цена — регистры, а от их числа зависит occupancy: сколько wavefront'ов CU держит одновременно, чтобы прятать задержки друг за другом.

Тайлы в LDS. Workgroup совместно загружает блоки `A` и `B` в LDS, после чего каждый

wavefront многократно читает их оттуда. Экономия глобальных загрузок оплачивается локальными обращениями, барьерами и возможными конфликтами банков LDS; раскладку тайла в LDS проверяйте по статистике конфликтов.

Скрытие задержек. Запрашивайте следующий тайл до того, как закончились вычисления над текущим: в регистры, во второй буфер LDS или через отдельные wavefront'ы-загрузчики. Смена индекса буфера сама по себе не доказывает перекрытия — убедитесь по ISA, что между загрузками и ожиданием их результата действительно выполняется арифметика.

Геометрия запуска. Размер workgroup'а, их число относительно восьми CU, порядок обхода выходных тайлов и постоянные (persistent) workgroup'ы, обходящие несколько тайлов, меняют и загрузку CU, и переиспользование данных в кэшах.

Матричные ядра. Инструкции MFMA с операндами FP32 выполняют за одну выдачу на порядки больше умножений, чем векторный FMA, но требуют особой раскладки операндов по lane. Будут ли они полезны, зависит от того, что вы узнали на шаге 5.

Две оговорки. Модель реализует не каждую инструкцию `gfx942`; если gem5 остановился на нереализованной инструкции, зафиксируйте это и измените подход, а не симулятор. И если результат выглядит как ошибка модели, опишите его воспроизводимо: найденный дефект модели — ценный результат работы, хотя рейтинг он не меняет.

Шаг 7 — Сдайте ядро, подготовьте отчёт и защиту

Сдайте на рейтинговый замер свой самый быстрый корректный вариант по правилам раздела 5.

Доведите вашу схему GPU до состояния, в котором по ней можно без текста проследить путь наивного и вашего итогового ядра и их данных. В отчёт включите:

- каталоги прогонов наивного и итогового ядер и разбор ISA обоих;
- причинные цепочки «HIP → ISA → блок модели → показатель» для наивного ядра;
- оценку пределов и диаграмму roofline из шага 5 с отмеченными на ней версиями;
- журнал экспериментов из шага 6;
- таблицу «наивное ядро против итогового» по рейтинговой метрике и основным показателям, в которой сдвиг каждого показателя объяснён микроархитектурно.

Работа готова, если каждый основной вывод можно устно связать с исходником, ISA, схемой GPU и наблюдаемой статистикой.

5. Правила соревнования

5.1 Задача и контракт

Нужно вычислить `C = A·B` для матриц `A` размера `256 x 128`, `B` размера `128 x 256` и `C` размера `256 x 256` в FP32. Все три матрицы хранятся в памяти GPU построчно (row-major), как в `hip-matmul`.

Сдаётся один файл `main.cpp`, построенный на основе `hip-matmul`. Хостовая часть сохраняет интерфейс образца: те же параметры командной строки, то же заполнение матриц, те же выделения памяти и копирования между хостом и GPU, поэлементную точную сверку с хостовым эталоном и строку `PASS` с контрольной суммой. Размеры можно сделать константами времени компиляции; на других размерах программа вправе завершиться с сообщением об ошибке, но не выдавать неверный результат.

Итоговую корректность проверяет преподаватель с `--seed` по своему выбору. Значения элементов — целые числа от −7 до 7, поэтому все произведения и частичные суммы представимы в FP32 точно, и результат не зависит от порядка суммирования.

5.2 Метрика и замер

Метрика — число GPU-активных тактов за прогон: `shaderActiveTicks` из `stats.txt`, делённое на период тактового сигнала GPU из `config.ini`. Как её получить, описано в Running the GPU lab. Она учитывает всю работу GPU за прогон — все ваши ядра и служебные ядра runtime'а, — поэтому всё, что вы запускаете на GPU, входит в результат. Меньше — лучше.

Рейтинговый замер выполняет преподаватель на backend'е `gem5-kmd` с конфигурацией модели по умолчанию и образами, выданными для работы: трижды, с медианой в качестве результата. Время одного и того же ядра от прогона к прогону немного колеблется, поэтому результаты, отличающиеся меньше чем на 2 %, считаются равными.

5.3 Что можно и чего нельзя

Можно менять всё, что исполняется на GPU, и то, как это запускается: число и устройство ядер, геометрию запуска, использование LDS и регистров, инструкции MFMA с операндами FP32, ассемблерные вставки и флаги компиляции своего файла.

Нельзя:

- менять симулятор, его конфигурацию и параметры запуска модели — число CU, кэши, частоту, политику распределения регистров;
- вычислять на хосте любую часть результата или переносить результаты между прогонами;
- переупорядочивать или преобразовывать входные матрицы на хосте: любая переупаковка данных выполняется на GPU и входит в метрику;
- понижать точность операндов (FP16, BF16, FP8, INT8 и подобные форматы) или

опираться на известные свойства данных — например, на то, что они целые и малы по модулю.

Последнее правило требует пояснения. Контрольные данные подобраны так, чтобы сверка была точной, и поэтому они же без потерь помещаются в узкие форматы — сверка такого нарушения не заметит, и преподаватель проверяет его по ISA. Переход к низкой точности — главный промышленный способ ускорения GEMM (раздел 2.3), но он меняет саму задачу: результат перестаёт совпадать с FP32 на произвольных данных. Если хотите исследовать его на модели, сделайте это отдельной главой отчёта с оценкой погрешности на случайных нецелых данных; в рейтинг такие версии не входят.

5.4 Ориентиры и оценивание

На рейтинговом размере наивное ядро выполняется примерно за 672 тысячи GPU-тактов. В каталоге примеров есть две оптимизированные реализации преподавателя: с LDS — около 300 тысяч тактов — и только на регистрах и кэшах — около 327 тысяч. Как они получены и что их ограничивает, описано в исследовании GEMM. Это ориентиры, которые можно и нужно обогнать. Изучать их не запрещено, но полезнее сделать это после собственных попыток: копия эталона в рейтинг не принимается, а на защите вы обосновываете каждое решение своими измерениями.

Для зачёта нужны выполненные шаги 1–7 и корректное ядро, заметно более быстрое, чем наивное, результат которого вы объяснили на защите. Место в рейтинге повышает оценку. Решение, устройство или ускорение которого автор объяснить не может, в рейтинг не включается.